Super

August 02, 2024

What is a JWT? Understanding JSON Web Tokens



Explore JWT tokens: secure, compact credentials for modern web authentication. Learn how they work, their benefits, and best practices for implementing them in your applications.



By Mostafa Ibrahim
Senior Software Engineer

Table of Contents

- Introduction
- What is a JWT?
- What are Tokens and Why Are They Needed?
- Structure of a JWT
- JWT Claim Convention
- How Do JWTs Work?

- Common Issues During Development
- Benefits of Using JWT Tokens
- Challenges and Considerations
- Implementing JWT Decoding with SuperTokens
- Further Reading Material

Introduction

Authentication and authorization are cornerstones of modern web security. They ensure that users are who they claim to be and that they can only access resources they are authorized for.

Traditional session-based authentication stores user information on the server. While this worked well for early monolithic apps, it introduced scaling challenges in distributed systems — where maintaining session state across multiple servers or microservices quickly became complex and inefficient.

JSON Web Tokens (JWTs) solve this by being **stateless**, **self-contained**, and easily verifiable. A JWT carries all necessary user information within the token itself, eliminating the need for frequent database lookups and enabling fast, scalable authentication across APIs, mobile apps, and microservices.

JWTs are most commonly used to **identify authenticated users**, issued by an authentication server and consumed by your APIs or frontend applications.

What is a JWT?

Super

Each JWT is digitally signed to prevent tampering and contains claims (pieces of information) about the user or session.

Here's an example JWT issued by Google during sign-in:


```
{
  "iss": "https://accounts.google.com",
  "azp": "1234987819200.apps.googleusercontent.com",
  "aud": "1234987819200.apps.googleusercontent.com",
  "sub": "10769150350006150715113082367",
  "email": "jsmith@example.com",
  "email_verified": true,
  "iat": 1353601026,
  "exp": 1353604926,
  "nonce": "0394852-3190485-2490358",
  "hd": "example.com"
}
```


This lets your app verify **who the user is**, **when the token was issued**, and **whether it's still valid** — all without calling Google's servers.

What are Tokens and Why Are They Needed?

If an authentication server simply sent user data as plain JSON, malicious users could modify fields (like the user ID) before sending it back to the server.

Tokens solve this problem by **encapsulating and cryptographically signing** data, ensuring the receiver can trust its authenticity.

There are two major token types:

- **Opaque tokens** — random identifiers referencing session data on the server.
- **JWTs** — self-contained tokens that include verifiable claims directly within the token.

Super

Structure of a JWT

A JWT has three parts, separated by dots (`.`):

1. Header

Defines the type (`JWT`) and the algorithm used to sign the token (e.g., `HS256`).

2. Payload

Contains the actual claims — such as user ID, expiration time, and roles.

3. Signature

Ensures integrity by signing the header and payload using a secret or public/private key pair.

```
header.payload.signature
```

JWT Claim Convention

JWTs follow standard claim conventions defined in RFC 7519:

- **iss** — Issuer (the authority generating the token)
- **sub** — Subject (user identifier)
- **aud** — Audience (the intended recipient app)
- **exp** — Expiration time

- **email**, **email_verified**, **roles** — Application-specific claims

Using standardized claims ensures interoperability across libraries and identity providers (e.g., Auth0, AWS Cognito, SuperTokens, Google Identity).

How Do JWTs Work?

JWT authentication typically follows this flow:

1. **User logs in** — Authentication server validates credentials.
2. **JWT issued** — Server signs and returns a JWT containing claims.
3. **Client stores token** — Usually in HttpOnly cookies or secure storage.
4. **Requests authenticated** — Client includes JWT in headers (e.g., `Authorization: Bearer <token>`).
5. **Server verifies JWT** — Using its secret or public key, checks signature validity and claim expiry.

This stateless model removes the need for a centralized session store, boosting scalability.

Pros and Cons of JWTs

✅ Advantages

- **Secure**: Cryptographically signed and tamper-proof.
- **Stateless**: No need for server-side session storage.
- **Cross-domain ready**: Works seamlessly for APIs, SPAs, and mobile apps.

Super

- **Difficult to revoke**: Tokens remain valid until expiration.
- **Key compromise risk**: A leaked secret allows attackers to forge tokens.
- **Size overhead**: Larger than opaque tokens due to embedded JSON.

For more on immediate revocation, see Revoking Access with a JWT Blacklist.

Common Use Cases for JWT Decoding

JWT decoding isn't just for curiosity — it's a vital tool in modern authentication.

1. Debugging Authentication and Authorization

Decoding tokens helps developers identify expired credentials, invalid issuers, or mismatched audiences — streamlining debugging during OAuth2 or OpenID Connect integrations.
(Verified by Auth0 Docs and RFC 7519).

2. Inspecting User Claims and Permissions

Decoded JWTs expose roles and scopes that control access within APIs or UIs — allowing developers to confirm if a user has the right permissions.
(Referenced by AWS Cognito Developer Guide).

3. Validating Client-Side Token Reception

SPAs and mobile apps often decode JWTs to verify `iss`, `aud`, or `exp` before making network requests, reducing unnecessary API calls.
(Supported by OWASP JWT Cheat Sheet).

synchronize sessions and user information.

(Based on OpenID Connect Core 1.0).

5. Edge and Mobile Use Cases

Edge services or mobile SDKs decode JWTs to optimize caching, routing, or offline validation of user sessions.

(Verified by Cloudflare Developer Docs).

Common Issues During Development

- **JWT Rejected** — Expired token or mismatched signature.
- **Invalid Scope** — App expects higher privileges than the token allows.
- **Decode Failed** — Malformed or incorrectly encoded JWT.

Always verify both the **structure** and **signature** before trusting a decoded JWT.

Benefits of Using JWT Tokens

- **Stateless Authentication** → Simplifies scaling and load balancing.
- **Compact and Portable** → Fits in headers, cookies, or query parameters.
- **Cross-Domain Compatibility** → Ideal for distributed microservice systems.
- **Customizable Claims** → Store user metadata securely.
- **Performance Boost** → Reduces I/O from database lookups.
- **Mobile and Offline Ready** → Works with limited connectivity.

Super

- Avoid storing sensitive data in the payload.
- Store JWTs in **HttpOnly cookies** instead of `localStorage`.
- Implement **token rotation** for added security.
- Rotate signing keys periodically.

Challenges and Considerations

Token Expiration and Refresh Strategy

Use **short-lived access tokens** with **longer-lived refresh tokens** for optimal balance between security and usability.

The refresh token (typically opaque) can safely regenerate JWTs when access tokens expire — as recommended in RFC 6749 OAuth2 Spec.

Secure Transport and Storage

Always use **HTTPS**, validate claims server-side, and prevent client-side tampering through secure storage mechanisms.

Implementing JWT Decoding with SuperTokens

SuperTokens simplifies JWT creation, validation, and rotation.

Integration Steps

3. Use its APIs to issue, verify, and decode JWTs — no custom logic required.

Why SuperTokens?

- ✅ **Open-source & self-hostable**
- ⚙️ **Zero-config JWT management**
- 🔐 **Automatic rotation & signature validation**
- 🌍 **Support for OAuth2, OIDC, and session revocation**

Try the SuperTokens JWT Encoder/Decoder Tool to inspect your own tokens securely.

Further Reading Material

- JWT, JWS, and JWE for Not-So-Dummies
- Revoking Access with a JWT Blacklist
- OpenID Connect & JWT Usage
- Best Way to Securely Manage User Sessions



Mostafa Ibrahim

Senior Software Engineer

Senior software engineer at a Fintech in London. Passionate about content creation, AI and traveling.

Super

Try SuperTokens in under 5 minutes

SuperTokens provides open source user authentication that is quick to implement and easy to customize.



Learn More

Product

Pricing

Product Roadmap

Self Hosted

Developers

Recipe/Implementation guides

API References

Features

Magic Link

SSO (Single-Sign On)

Multi-Tenant

Account Linking

Attack Protection Suite

Social Login

Passwordless Login

Email Password Login

Super

Blog

Careers

Customers

Community

Engineering

Launches

Security at SuperTokens

Contribute

Contact Sales

Terms of Service

Privacy Policy

API Status

Auth0 vs SuperTokens vs Cognito

JWT Decoder

Super

©SuperTokens 2026



View Report